



交换排序

后续关注公众号「研木」
获取更多学习资料

冒泡排序

快速排序





交换排序

冒泡排序

快速排序

基本思想

从前往后(反过来也行)依次比较相邻元素，如果逆序则交换，不断循环到有序

逻辑演示

已有序序列

13

27

38

49

49

65

76

97

对所有元素进行过一遍处理称为一趟
冒泡排序可以进行优化
如果一趟中未发生元素交换，可以直接结束排序

是稳定的排序算法





基本思想

从前往后(反过来也行)依次比较相邻元素，如果逆序则交换，不断循环到有序

```
void BubbleSort(SqList &L)
```

```
{ // 对顺序表 L 进行冒泡排序
```

```
    m=L.length-1;
```

```
    flag=1;
```

```
    //如果本趟排序没有发生交换，则不会执行下一趟排序
```

```
    while((m>0)&&(flag==1)){
```

传入待排序数组

m>0不成立
跳过while

```
        flag=0; //flag用来标记某一趟排序是否发生交换
```

```
        for(j=1;j<=m;j++){
```

```
            if(L[j]>L[j+1]){
```

```
                flag=1; //表示本趟排序发生了交换
```

```
                t=L[j];
```

```
                L[j]=L[j+1];
```

```
                L[j+1]=t;
```

```
            }
```

```
        } //for
```

```
        --m;
```

```
    } //while
```

```
} //BubbleSort
```

j=2

flag=1

L.length=4

m=0

存储内容

-

15

27

29

38

数组下标

0

1

2

3

4

排序完成

平均时间复杂度 $O(n^2)$

空间复杂度 $O(1)$

稳定排序

也适用于链式存储



交换排序

冒泡排序

快速排序

基本思想

1. 选择一个元素作为枢纽（选择表中第一个元素）

2. 让枢纽左边存储小于枢纽的元素，右边存储大于枢纽的元素

具体做法：表左右侧设立`low`, `high`指针，初始`low=1`; `high=L.length`

`high`指针往左（小）走，碰到比枢纽元素小的就把元素放到`low`位置，然后操作`low`指针

`low`指针往右（大）走，碰到比枢纽元素大的就把元素放到`high`位置，然后操作`high`指针





```
void QSort(SqList &L,int low,int high)
{ //调用前置初值: low=1; high=L.length;
  // 对顺序表 L 中的子表 L[low..high] 进行快速排序
  if(low<high)
  { //长度大于1
    pivotloc=Partition(L, low, high);
    //将L[low..high]一分为二, pivotloc是枢轴位置
    QSort(L, low, pivotloc-1); //对左子表递归排序
    QSort(L, pivotloc+1, high); //对右子表递归排序
  }
}
```

```
int Partition(SqList &L, int low, int high)
{ //对顺序表 L 中的子表 L[low..high] 进行一趟排序, 返回枢轴位置
  L[0] = L[low]; // 用子表的第一个记录作为枢轴记录
  pivotkey = L[low]; // 枢轴记录关键字保存在 pivotkey中
  while (low < high){
    while (low < high && L[high] >= pivotkey) --high;
    L[low] = L[high]; //将比枢轴记录小的记录移到低端
    while (low < high && L[low] <= pivotkey) ++low;
    L[high] = L[low]; //将比枢轴记录大的记录移到高端
  } //while
  L[low] = L[0]; //枢轴记录到位
  return low; //返回枢轴位置
}
```

逻辑演示

每次选择枢轴元素时, 理想状态下是做到最平衡的划分
即左右两个子问题大小都 $\leq \frac{n}{2}$, 此时时间复杂度为 $O(n \log_2 n)$

平均情况下运行时间与最佳情况下
运行时间接近最坏情况是元素已整体有序或基本逆序时, 时间复杂度为 $O(n^2)$

存储内容

76

13

27

38

49

49

65

76

97

数组下标

0

1

2

3

4

5

6

7

8

排序完成
快排是不稳定算法
(这里的例子只是运气好稳定了)

平均情况下空间复杂度 $O(\log_2 n)$
最好情况下空间复杂度为 $O(\log_2 n)$
最坏情况下 $n-1$ 次递归, 空间复杂度 $O(n)$

平均情况下时间复杂度 $O(n \log_2 n)$
最好情况下时间复杂度为 $O(n \log_2 n)$
最坏情况时间复杂度 $O(n^2)$



```
void QSort(SqList &L,int low,int high)
{ //调用前置初值: low=1; high=L.length;
  // 对顺序表 L 中的子表 L[low..high] 进行快速排序
  if(low<high)
  { //长度大于1
    pivotloc=Partition(L, low, high);
    //将L[low..high]一分为二, pivotloc是枢轴位置
    QSort(L, low, pivotloc-1); //对左子表递归排序
    QSort(L, pivotloc+1, high); //对右子表递归排序
  }
}
```

```
int Partition(SqList &L, int low, int high)
{ //对顺序表 L 中的子表 L[low..high] 进行一趟排序, 返回枢轴位置
  L[0] = L[low]; // 用子表的第一个记录作为枢轴记录
  pivotkey = L[low]; // 枢轴记录关键字保存在 pivotkey中
  while (low < high){
    while (low < high && L[high] >= pivotkey) --high;
    L[low] = L[high]; //将比枢轴记录小的记录移到低端
    while (low < high && L[low] <= pivotkey) ++low;
    L[high] = L[low]; //将比枢轴记录大的记录移到高端
  } //while
  L[low] = L[0]; //枢轴记录到位
  return low; //返回枢轴位置
}
```

逻辑演示

存储内容

76

13

27

38

49

49

65

76

97

数组下标

0

1

2

3

4

5

6

7

8

标准的快排依赖于随机存储的特性
主要适用于顺序存储

